

08 API - Unit Testing

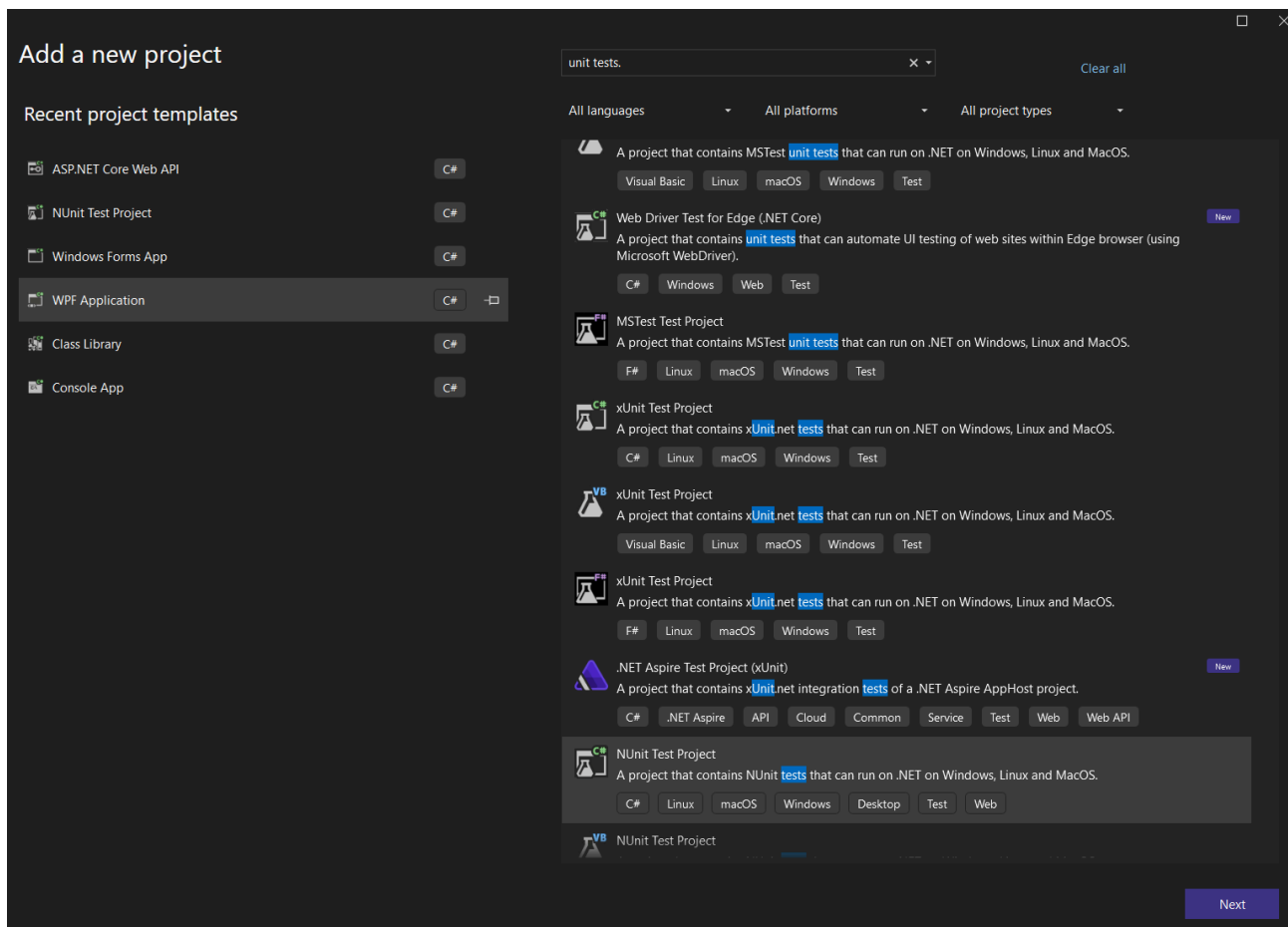
📅 09-09-2026 ⌚ Ongeveer 20 minuten 🗨️ Ongeveer 3072 woorden

Herhaling Unit Testing

Een unit test is niet meer dan een methode in een testproject dat individueel uitgevoerd kan worden zonder de volledige applicatie te runnen. Het is een stukje geautomatiseerde code dat een methode uitvoert en controleert of het resultaat van die methode overeenkomt met wat je verwacht. Een unit test bestaat bijna altijd uit drie onderdelen: Arrange, Act en Assert. In het Arrange-gedeelte verzamel je alle data en andere benodigdheden die de test nodig heeft. In het Act-gedeelte voer je de methode uit die je wilt testen. Ten slotte controleer je in het Assert-gedeelte of het resultaat van de methode klopt met de verwachtingen.

Nunit toevoegen

Voeg een nieuw project toe aan je solution en zoek naar unit tests. Zoals je ziet, zijn er verschillende testing frameworks beschikbaar. Het is hierbij belangrijk om niet gefocust te zijn op het uit je hoofd leren van code of frameworks, maar om de onderliggende concepten te begrijpen. De syntax kan per framework verschillen, maar de concepten zijn vrijwel altijd hetzelfde. Het maakt dus niet uit welk framework je kiest, zolang je de concepten maar begrijpt. In deze cursus kiezen we voor het NUnit-framework.



Noem je nieuwe project bijvoorbeeld WebApiDemo.Tests. Er wordt standaard een nieuwe testklasse voor je aangemaakt. Zoals je ziet, is dit een gewone C#-klasse met een methode waarboven [Test] tussen vierkante haken staat. Als je dit testattribuut toevoegt aan een publieke void-methode, zal deze gemarkeerd worden als een unit test en kun je deze zien in je Test Explorer. Als je je Test Explorer niet kunt vinden, kun je deze eenvoudig terugvinden via de zoekbalk bovenaan in Visual Studio.

Een unit test kan kinderlijk eenvoudig zijn. Zo kun je bijvoorbeeld een test opstellen die controleert of 2 plus 3 gelijk is aan 5.

```

1  using NUnit.Framework;
2
3  namespace Project.Tests
4  {
5      public class SimpleMathTests
6      {
7          [Test]
8          public void Addition_TwoPlusThree_EqualsFive()
9          {
10             // Arrange
11             int num1 = 2;

```

```
11         int num2 = 3;
12
13         // Act
14         int result = num1 + num2;
15
16         // Assert
17         Assert.That(result, Is.EqualTo(5));
18     }
19 }
20 }
21 }
```

Zoals je ziet, verzamel je eerst de benodigde variabelen, daarna voer je een bepaalde actie uit, in dit geval een optelsom, en uiteindelijk controleer je of het resultaat van die actie overeenkomt met wat je verwacht.

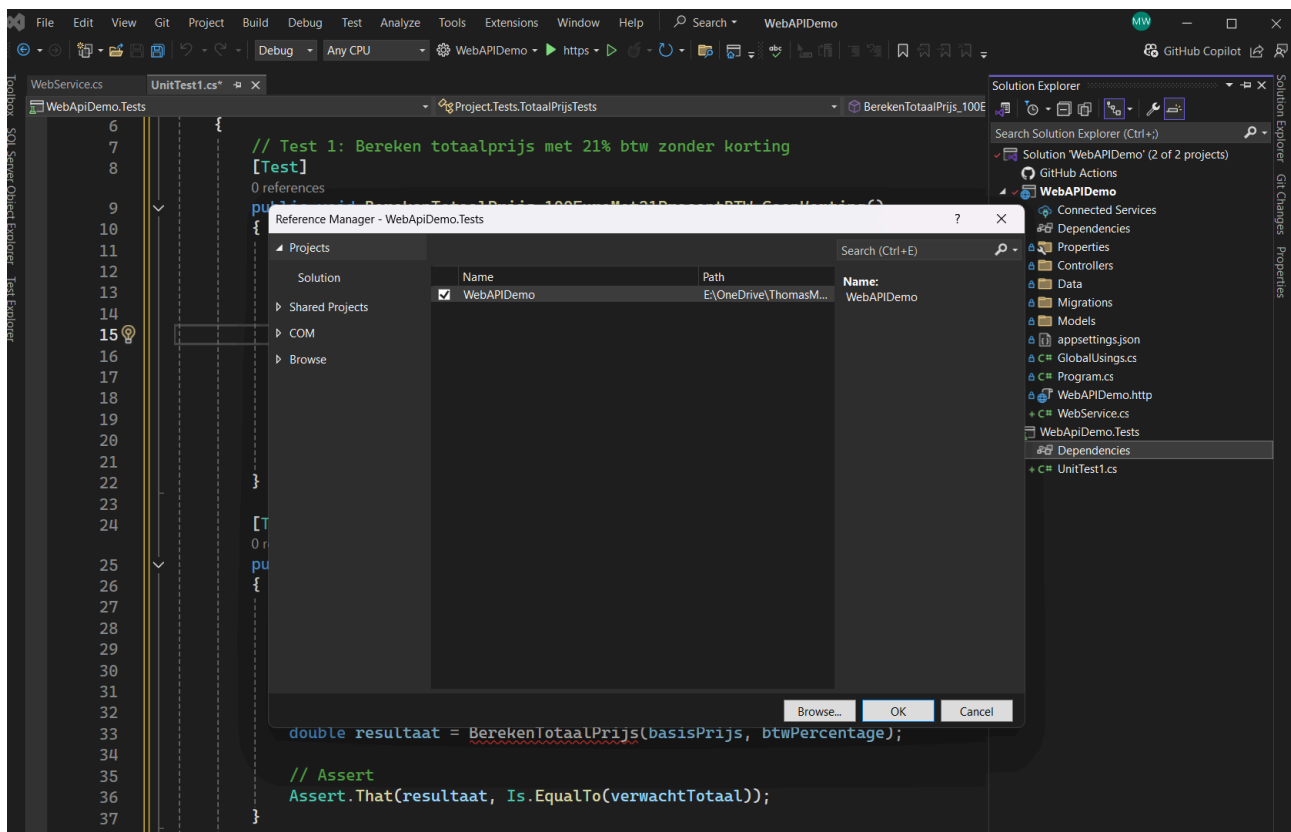
Deze test zegt natuurlijk niets over de algemene staat van de applicatie. Het controleert alleen een lokale bewerking. Dit soort tests voer je dan ook niet uit in een echte applicatie. In plaats daarvan test je methoden uit andere klassen om te controleren of ze correct functioneren binnen hun specifieke context.

Nota: In een API of MVC project is het niet mogelijk om Controller klassen te testen. Hierom plaats je doorgaans de merendeel van je business logica in Services of Helper klassen en voer je daarop je Unit Tests uit.

Project referentie toevoegen

Meestal wil je businessklassen testen, zoals services en helperklassen. Aangezien deze klassen zich buiten je testproject bevinden, moet je een referentie naar het project toevoegen dat je wilt testen. Volg deze stappen om dit te doen:

- Voeg een projectreferentie toe: Rechtsklik op de node "Dependencies" van je testproject in de Solution Explorer. Selecteer "Add Project Reference". Selecteer het project:
- Vink het/de project(en) aan dat je wilt testen. Na het toevoegen van de projectreferentie kunnen de publieke klassen en methoden uit het gerefereerde project gebruikt worden in je testproject.



```

1 namespace Project.Tests
2 {
3     public class SimpleMathTests
4     {
5         [Test]
6         public void Addition_Service_TwoPlusThree_EqualsFive()
7         {
8             // Arrange
9             int num1 = 2;
10            int num2 = 3;
11            var mathOperations = new MathOperations();
12
13            // Act
14            // Nu wordt er een methode in een externe klasse getest.
15            int result = _mathOperations.Add(2, 3);
16
17            // Assert
18            Assert.That(result, Is.EqualTo(5));
19        }
20    }
21 }

```

csharp

Test Driven Design

In Test Driven Design (TDD) schrijf je je tests voordat je ook maar één regel code schrijft. Dit klinkt misschien verwarrend: hoe kun je nu een test schrijven voor een methode die nog niet bestaat? Maar eigenlijk is het heel eenvoudig en het helpt je zelfs om eenvoudiger en beter testbare code te schrijven.

Beeld je het volgende scenario in: je wordt gevraagd om een methode te schrijven die de totaalprijs van een aankoop berekent. Soms moet er 21% btw worden berekend, soms 6%. Als de totale prijs zonder btw meer dan € 1.000 bedraagt, wordt er 10% korting op de totaalprijs verrekend.

Je weet nog niet hoe je de methode precies gaat opstellen, maar je kent de businessvereisten wel. Je kunt deze dus al vertalen naar unit tests, nog voordat je met de methode(s) begint. Het is geen probleem dat de methode die je wilt testen nog niet bestaat. Schrijf de test gewoon alsof de methode al bestaat, met alle benodigde input- en outputparameters, en laat Visual Studio de methode later voor je genereren. Op deze manier kan je je volledig op je test concentreren zonder rekening te moeten houden met reeds bestaande code.

Test Driven Design toepassen

TDD bestaat uit vijf stappen:

- Stel een unit test op: In deze stap schrijf je de volledige unit test die de gevraagde businesslogica zal controleren.
- Run de test en valideer dat deze faalt: Deze stap wordt soms over het hoofd gezien, maar is van cruciaal belang. Aangezien de businesslogica nog niet bestaat, moet deze test falen. Als dat niet gebeurt, is er iets fundamenteel mis met de applicatie.
- Schrijf de code totdat alle tests slagen: In deze stap schrijf je de code die nodig is om je test te laten slagen. De code hoeft nog niet perfect te zijn, zolang alle tests maar groen kleuren.
- Optimaliseer en refactor je code: Verfijn je code zodat deze van hoge kwaliteit is en zorg ervoor dat er geen overbodige code meer aanwezig is.
- Herhaal: Begin opnieuw met een nieuwe unit test en herhaal dit proces totdat alle gevraagde businesslogica in je applicatie is geïmplementeerd.

Als je je code op deze manier schrijft, zorg je ervoor dat de code direct van goede kwaliteit is en dat alles correct werkt, omdat alle vereisten in je unit tests zijn opgenomen. Daarnaast dwing je jezelf, omdat je je eerst op de unit tests richt om methodes te schrijven die eenvoudiger en beter testbaar zijn. Dit voorkomt onnodige complexiteit.

```
1  using NUnit.Framework;
2  using WebAPIDemo;
3
4  namespace Project.Tests
5  {
6      public class TotaalPrijsTests
7      {
8          WebService _myService;
9
10         [SetUp]
11         public void Setup()
12         {
13             // Setup wordt uitgevoerd voor elke test
14             _myService = new WebService();
15         }
16
17
18         // Test 1: Bereken totaalprijs met 21% btw zonder korting
19         [Test]
20         public void
21         BerekenTotaalPrijs_100EuroMet21ProcentBTW_GeenKorting()
22         {
23             // Arrange
24             double basisPrijs = 100;
25             double btwPercentage = 0.21;
26             double verwachtTotaal = 121;
27
28
29             // Act
30             double resultaat = _myService.BerekenTotaalPrijs(basisPrijs,
31             btwPercentage);
32
33             // Assert
34             Assert.That(resultaat, Is.EqualTo(verwachtTotaal));
35         }
36
37
38         // Test 2: Bereken totaalprijs met 6% btw zonder korting
```

csharp

```
38         [Test]
39         public void
40         BerekenTotaalPrijs_100EuroMet6ProcentBTW_GeenKorting()
41         {
42             // Arrange
43             double basisPrijs = 100;
44             double btwPercentage = 0.06;
45             double verwachtTotaal = 106;
46
47             // Act
48             double resultaat = _myService.BerekenTotaalPrijs(basisPrijs,
49             btwPercentage);
50
51             // Assert
52             Assert.That(resultaat, Is.EqualTo(verwachtTotaal));
53         }
54
55
56         // Test 3: Bereken totaalprijs met 21% btw en 10% korting (prijs
57 > €1.000)
58         [Test]
59         public void
60         BerekenTotaalPrijs_1200EuroMet21ProcentBTW_MetKorting()
61         {
62             // Arrange
63             double basisPrijs = 1200;
64             double btwPercentage = 0.21;
65             double verwachtTotaal = 1166.40;
66
67             // Act
68             double resultaat = _myService.BerekenTotaalPrijs(basisPrijs,
69             btwPercentage);
70
71             // Assert
72             Assert.That(resultaat, Is.EqualTo(verwachtTotaal));
73         }
74
75         // Test 4: Bereken totaalprijs met 6% btw en 10% korting (prijs >
76 €1.000)
77         [Test]
78         public void
79         BerekenTotaalPrijs_1100EuroMet6ProcentBTW_MetKorting()
80         {
```

```
81         // Arrange
82         double basisPrijs = 1100;
83         double btwPercentage = 0.06;
84         double verwachtTotaal = 1058.40;
85
86         // Act
87         double resultaat = _myService.BerekenTotaalPrijs(basisPrijs,
88         btwPercentage);
89
90         // Assert
91         Assert.That(resultaat, Is.EqualTo(verwachtTotaal));
92     }
93 }
94 }
```

Eenmaal de tests geschreven zijn en falen kunnen we makkelijk de methode onder test laten genereren door Visual Studio. Aangezien we onze methode hebben geschreven in functie van onze test en niet andersom zal deze methode al de nodige in- en output parameters reeds voorzien. Methodes met input en output zijn zeer doorgaans makkelijk te testen, volds zijn vaak moeilijker.

Services

Een service zal steeds bestaan uit een interface en een implementerende klasse. Deze werking werkt gelijkaardig zoals we repositories behandeld hebben. Binnen een service zullen methodes voorzien worden om business logica uit te voeren. Bijvoorbeeld wiskundige berekeningen (Prijs inclusief berekenen, korting toekennen, ...) of andere technieken om met gegevens overweg te gaan (Het exporteren of importeren van excel-bestanden).

```
1 namespace WebAPIDemo
2 {
3     public interface IWebService
4     {
5         double BerekenTotaalPrijs(double basisPrijs, double
6         btwPercentage);
7     }
8 }
```

```
1 public double BerekenTotaalPrijs(double basisPrijs, double btwPercentage)
2 {
3     throw new NotImplementedException();
4 }
```

```
1 namespace WebAPIDemo
2 {
3     public class WebService : IWebService
4     {
5         public double BerekenTotaalPrijs(double basisPrijs, double
6 btwPercentage)
7         {
8             double prijsMetBTW = basisPrijs * (1 + btwPercentage);
9
10            if (basisPrijs > 1000)
11            {
12                prijsMetBTW *= 0.90;
13            }
14
15            return Math.Round(prijsMetBTW, 2);
16        }
17    }
18 }
```

csharp

TestCase

Bij methodes die wiskundige operaties uitvoeren, wil je waarschijnlijk je methode testen met een breed scala aan invoerwaarden. Zo is het bijvoorbeeld eenvoudig om te testen met ronde getallen, zoals we hierboven deden, maar je wilt ook zeker weten dat komma-getallen correct worden verwerkt.

Je kunt hiervoor verschillende testmethodes aanmaken, maar dat kan snel leiden tot duplicate code. Het is veel efficiënter om één testmethode opnieuw uit te voeren met verschillende invoerwaarden. Dit kun je eenvoudig bereiken door parameters toe te voegen aan je testmethode en het [Test]-attribuut te vervangen door [TestCase]. Je kunt dan meerdere [TestCase]-attributen toevoegen aan dezelfde testmethode, elk met zijn eigen invoerwaarden. Op deze manier kun je eenvoudig grensgevallen en verschillende scenario's testen.

```
1  using NUnit.Framework;
2  using WebAPIDemo;
3
4  namespace Project.Tests
5  {
6      public class TotaalPrijsTests
7      {
8          private WebService _myService;
9
10         [SetUp]
11         public void Setup()
12         {
13             // Setup wordt uitgevoerd voor elke test
14             _myService = new WebService();
15         }
16
17         // Test 1 en 2: Bereken totaalprijs met verschillende BTW-
18         // percentages zonder korting
19         [TestCase(100, 0.21, 121)] // €100 met 21% btw zou €121 moeten
20         // zijn
21         [TestCase(100, 0.06, 106)] // €100 met 6% btw zou €106 moeten
22         // zijn
23         public void BerekenTotaalPrijs_BtwZonderKorting(double
24         basisPrijs, double btwPercentage, double verwachtTotaal)
25         {
26             // Act
27             double resultaat = _myService.BerekenTotaalPrijs(basisPrijs,
28             btwPercentage);
29
30             // Assert
31             Assert.That(resultaat, Is.EqualTo(verwachtTotaal));
32         }
33
34         // Test 3 en 4: Bereken totaalprijs met korting toegepast
35         [TestCase(1200, 0.21, 1166.40)] // €1200 met 21% btw minus 10%
36         // korting zou €1166,40 moeten zijn
37         [TestCase(1100, 0.06, 1058.40)] // €1100 met 6% btw minus 10%
38         // korting zou €1058,40 moeten zijn
39         public void BerekenTotaalPrijs_BtwMetKorting(double basisPrijs,
40         double btwPercentage, double verwachtTotaal)
41         {
42             // Act
43             double resultaat = _myService.BerekenTotaalPrijs(basisPrijs,
```

csharp

```
        btwPercentage);

        // Assert
        Assert.That(resultaat, Is.EqualTo(verwachtTotaal));
    }
}
```

Mocking

Het komt vaak voor dat klassen getest moeten worden die dependencies hebben. Stel je bijvoorbeeld een methode voor in een service die een bestelling afrondt. Deze service heeft een dependency van een klasse zoals ProductRepository, die gegevens uit de database ophaalt, en een Logger, die bijhoudt wat er precies gebeurt. In de daadwerkelijke code worden deze dependencies aangemaakt door het Dependency Injection Framework. Tijdens unit tests heb je deze luxe niet en dus moeten de dependencies zelf worden gecreëerd.

```
1  public interface IOrderService                                csharp
2  {
3      Order CompleteOrder(int productId)
4  }
```

```
1  public class OrderService : IOrderService                    csharp
2  {
3      private readonly IProductRepository _productRepository;
4      private readonly ILogger _logger;
5
6      public OrderService(IProductRepository productRepository, ILogger
7  logger)
8      {
9          _productRepository = productRepository;
10         _logger = logger;
11     }
12
13     public Order CompleteOrder(int productId)
14     {
15         var product = _productRepository.GetProductById(productId);
16         Order order = new Order();
17     }
```

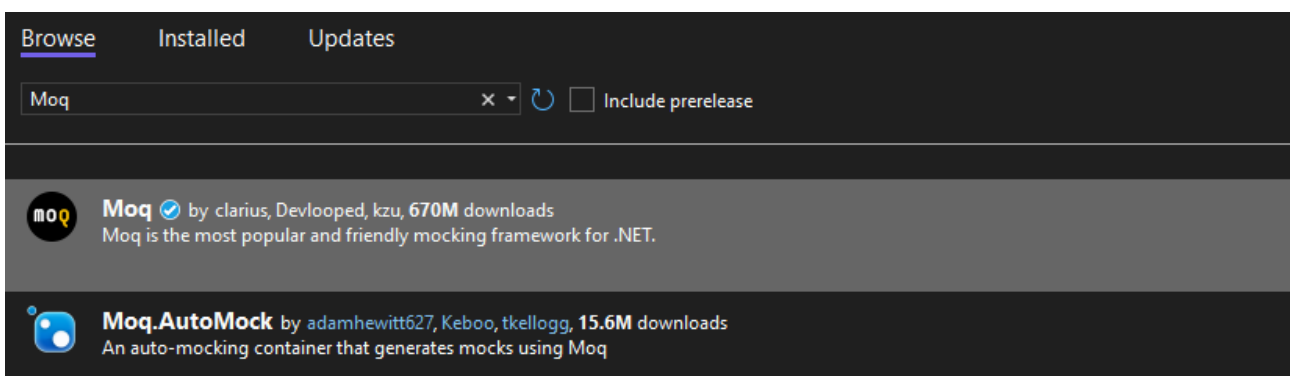
```
18         order.Product = product;
19         order.TransportPrijs = BerekenTransport(product.Gewicht);
20         order.VerzendDatum = DateTime.Now().Days.Add(2);
21         order.IsComplete = true;
22
23         _logger.Log($"Order completed for product ID: {productId}");
24         return order;
25     }
}
```

Het is belangrijk om deze dependencies niet daadwerkelijk aan te maken. Een van de voordelen van Dependency Injection frameworks is namelijk dat complexe creatielogica wordt vermeden. Daar wil je niet opniew naartoe. Daarnaast zijn vaak bepaalde dependencies niet nodig bij het testen van een specifieke methode. Tijdens het Unit Testen ben je bijvoorbeeld niet geïnteresseerd in het loggen van data, je bent louter de functionaliteit aan het testen om een bestelling af te ronden. Bovendien communiceert de ProductRepository-klasse met een database. Tijdens unit tests is het niet wenselijk om een echte database te gebruiken, omdat data volatiel is en het moeilijk te voorspellen valt hoe deze er in de toekomst uit zal zien. Ook is het ongewenst dat de database telkens wordt gevuld met testdata bij elke unit test.

Om deze redenen worden dependencies in unit tests doorgaans gemoct. Nepversies worden aangemaakt die net echt genoeg zijn om C# te laten denken dat ze authentiek zijn, zodat de code getest kan worden. Dit kan vergeleken worden met een vogelverschrikker op een pas gezaaid veld: het is geen echte persoon, maar een voldoende realistische representatie om de vogels af te schrikken. In deze cursus zal je gebruik maken van het Moq framework om dependencies te mocken.

MOQ

Vooraleer je Moq kan gebruiken moet je dit installeren in je Unit Test project. Dit gebeurt weerom via de NuGet Package Manager.



Als Moq geïnstalleerd is, kan je de dependencies mocken. Dit doe je door de klassen in een `Mock<T>` container te wrappen. Je kan dan, in de test, je `mock.Object` als input meegeven in de constructor van de service.

```
1  using Moq;
2  using NUnit.Framework;
3
4  public class OrderServiceTests
5  {
6      [Test]
7      public void CompleteOrder_SetsCompleteToTrue()
8      {
9          // Arrange
10         var mockProductRepository = new Mock<IProductRepository>();
11         var mockLogger = new Mock<ILogger>();
12         var orderService = new OrderService(mockProductRepository.Object,
13 mockLogger.Object);
14
15         // Act
16         var order = orderService.CompleteOrder(testProductId);
17
18         // Assert
19         Assert.IsTrue(order.IsComplete)
20     }
21 }
```

Moq Setups

Op bovenstaande manier zal je in je Unit Test niet langer build errors krijgen, maar zal de test nog steeds falen. Als je een kijkje neemt naar de `CompleteOrder` zie je dat deze methode afhankelijk is van een `Repository` klasse die data uit de database ophaalt. Deze klasse is nu een `Mock` en zal nu natuurlijk geen data returnen. Het is uiteindelijk geen echte klasse. Maar uiteindelijk heb je wel de data nodig uit de methode die aangeroepen werd. Hoe kan je dit oplossen? Een oplossing is het gebruik van de `Setup` expressie. Met `Setup` kan je een gemockte methode hard gecodeerde return values geven als het aangeroepen wordt waardoor je verder kan met je tests.

```
1  using Moq;
2  using NUnit.Framework;
3
4  public class OrderServiceTests
5  {
6      [Test]
7      public void CompleteOrder_LogsCorrectMessage()
8      {
9          // Arrange
10         var mockProductRepository = new Mock<IProductRepository>();
11         var mockLogger = new Mock<ILogger>();
12
13         // Met setup kan je een mock methode hard gecodeerde data laten
14         returnen
15         mockProductRepository
16             .Setup(repo => repo.GetProductById(1))
17             .Returns(new Product
18             {
19                 Id = 1,
20                 Gewicht = 22,
21             });
22
23         var orderService = new OrderService(mockProductRepository.Object,
24 mockLogger.Object);
25
26         // Act
27         orderService.CompleteOrder(testProductId);
28
29         // Assert
30         mockLogger.Verify(logger => logger.Log($"Order completed for
product ID: {testProductId}"), Times.Once);
    }
}
```

Merk op dat de GetProductById methode nu moet aangeroepen worden met de value '1'. Als je een andere value meegeeft zal de hard gecodeerde setup niet getriggerd worden. Als het niet uitmaakt welke input aan de methode meegegeven moet worden, maar het gewoon eender welke int value kan zijn kan je de input veranderen door `It.IsAny<T>()`;

```
1  using Moq;
2  using NUnit.Framework;
3
4  public class OrderServiceTests
5  {
6      [Test]
7      public void CompleteOrder_LogsCorrectMessage()
8      {
9          // Arrange
10         var mockProductRepository = new Mock<IProductRepository>();
11         var mockLogger = new Mock<ILogger>();
12
13         // Met setup kan je een mock methode hard gecodeerde data laten
14         returnen
15         mockProductRepository
16             .Setup(repo => repo.GetProductById(It.IsAny<int>())) // Via de
17             It.IsAny<T> kan je een mock methode aanroepen met eender welke value van
18             type T (bv, 1, 5 ,22, ...)
19             .Returns(new Product
20             {
21                 Id = 1,
22                 Gewicht = 22,
23             });
24
25         var orderService = new OrderService(mockProductRepository.Object,
26         mockLogger.Object);
27
28         // Act
29         orderService.CompleteOrder(testProductId);
30
31         // Assert
32         mockLogger.Verify(logger => logger.Log($"Order completed for
33         product ID: {testProductId}"), Times.Once);
34     }
35 }
```

AutoMapper als dependency?

Gebruikt je service AutoMapper als dependency? Dan kan je dit niet mocken. Deze dependency zal je manueel moeten aanmaken met een lokaal gegenereerd Configuration bestand. Gebruik onderstaande voorbeeld om een AutoMapper als dependency toe te voe-

gen bij het aanroepen van een methode uit een Controller.

```
1          // MapperProfile staat geconfigureerd in een niet-test    csharp
2  project
3          var config = new MapperConfiguration(cfg => {
4  cfg.AddProfile(new MyMapperProfile()); });
5          IMapper mapper = new Mapper(config);

          ProductController ctx = new ProductController(mapper);
```

Een echte unit test met dependency(ies) en AutoMapper zou er als volgt uit kunnen zien, inclusief Controller en Service: Merk op dat we hier een Service testen (business klasse) en niet een Controller of een Repository. Repositories testen we doorgaans niet omdat data volatiel is en een Controller moeilijk te unit testen is omwille van de return types. Hierdoor blijven Services als ideale kandidaat over. Daarom kiezen we er liefst zoveel mogelijk voor om Business logica binnenin een Service te plaatsen.

```
1  // Service    csharp
2  public interface IProductService
3  {
4      Task<Product> GetProductAsync(int id)
5  }
```

```
1  // Service    csharp
2  public class ProductService : IProductService
3  {
4      private IProductRepo _repo;
5      private IMapper _mapper;
6
7      public ProductService(IProductRepo repo, IMapper mapper)
8      {
9          _repo = repo;
10         _mapper = mapper;
11     }
12
13     public async Task<Product> GetProductAsync(int id)
14     {
15         ProductEntity entity = await
16         _repo.GetProductWithCategory(id);
17
18         Product model = _mapper.Map<Product>(entity);
```

```
19         // We voeren berekeningen ENKEL op het Model uit
20         model.TotaalPrijs = model.Prijs + 15;
21
22         return model;
23     }
24 }
```

```
1 // Controller csharp
2 public class ProductController : ControllerBase
3 {
4     private IProductService _service;
5     private IMapper _mapper;
6
7     public ProductController(IProductService service, IMapper mapper)
8     {
9         _service = service;
10        _mapper = mapper;
11    }
12
13    [HttpGet("id")]
14    public async Task<ActionResult<ProductDTO[]>> GetProductAsync(int
15 id)
16    {
17        Product model = await _service.GetProductAsync(id);
18
19        ProductDTO result = _mapper.Map<ProductDTO>(model);
20
21        if (result != null)
22            return Ok(result);
23        else
24            return NotFound();
25    }
26 }
```

```
1 // Unit Test csharp
2 public class ProductTests
3 {
4     [Test]
5     public async Task
6 AlsProductOpgehaaldWordt_WordenPrijsenCorrectBerekend()
7     {
```

```
8      // Arrange
9      ProductEntity mockProduct = new ProductEntity
10     {
11         Prijs = 100,
12         Naam = "TestProduct"
13     };
14
15     // Mock Dependency
16     Mock<IProductRepo> mockRepo = new Mock<IProductRepo>();
17     mockRepo.Setup(x => x.GetProductWithCategory(It.IsAny<int>
18     ())).ReturnsAsync(mockProduct);
19
20     // Set Automapper
21     var config = new MapperConfiguration(cfg => { cfg.AddProfile(new
22     MapperProfile()); });
23     IMapper mapper = new Mapper(config);
24
25     var service = new ProductService(mockRepo.Object, mapper);
26
27     // Act
28     var product = await service.GetProductAsync(1);
29
30     // Assert
31     Assert.That(product.TotaalPrijs, Is.EqualTo(115));
32 }
33 }
```

Kortom - Voordelen van Unit Testing

Wat zijn de voordelen van unit testing? Voor een gebruiker zijn er op het eerste gezicht niet veel. Het zal je applicatie niet sneller of veiliger maken. Maar voor een developer biedt unit testing talloze voordelen.

- Snel overzicht over de gezondheid van je applicatie: Als je voldoende unit tests hebt en deze allemaal groen kleuren, kun je er redelijkerwijs van uitgaan dat je applicatie werkt zoals verwacht.
- Documentatie: De meeste developers schrijven niet graag uitgebreide documentatie, waardoor het vaak onduidelijk is wat een applicatie precies zou moeten doen. Unit tests kunnen hier enorm bij helpen, omdat ze expliciet beschrijven wat er van een bepaalde functie of methode wordt verwacht. Wanneer je gebruikmaakt van duidelijke

methodenamen, vertelt de test naam al direct wat er hoort te gebeuren, zonder dat je de code hoeft te lezen.

- Eenvoudiger refactoren: Met goede unit tests word je onmiddellijk op de hoogte gebracht als je, al dan niet onbewust, het verwachte resultaat van een methode hebt gebroken. Hierdoor kun je eventuele fouten direct herstellen of aanpassen. Dit geeft je het vertrouwen om bestaande code met zekerheid te refactoren of te verbeteren, zonder dat je bang hoeft te zijn om onbewust fouten te (her)introduceren.
- Betere code: Het schrijven van unit tests moedigt je aan om je code te ontwerpen met testbaarheid in gedachten. Dit leidt vaak tot een betere en meer modulaire architectuur, omdat je geneigd zal zijn om kleine, duidelijke functies te schrijven die makkelijk te testen zijn.
- Snellere foutopsporing, kostenbesparing op de lange termijn.

[Hier vind je het WebAPIDemo-project](#)

Laatst bijgewerkt:09-09-2026, 08:52

Bijdragers: JorenSynaeveTM